



M257 Exam Handbook

Important: This Exam Handbook may be taken into the Examination but it must not contain any annotations or any other additions except for any official Errata that appear on the M257 website.

The materials are drawn chiefly from the course units and Java APIs. Although this handbook includes material not discussed in M257, because we may wish to refer to such material in an exam, this does not imply a need to study details outside of the course materials.

This booklet is not designed to be read from cover to cover; rather you should use its table of contents to find the documentation for a particular method, class or interface. The information presented has generally been abbreviated.

Methods and constructors are public unless otherwise indicated.

Contents

1	SOME JAVA SYNTAX	2
1.0	Keywords	2
1.1	Operators	2
1.2	Iteration	3
1.3	Selection	4
1.4	Arrays	5
1.5	Exceptions	5
2	JAVA API EXCERPTS	6
2.0	Errors and exceptions	6
2.1	Some common methods of the <code>Thread</code> class	7
2.2	Some common methods of the <code>String</code> class	8
2.3	Some important interfaces	9
2.4	Collection classes	11
2.5	Input and output streams and related classes	11
2.6	Java Swing classes	18
2.7	Event handling	22
2.8	Layout managers	23
2.9	Graphics	24
2.10	Some common methods of the <code>MIDlet</code> class	24

1 Some Java syntax

1.0 Keywords

abstract	case	default	final	if	long	return	this
boolean	catch	do	finally	implements	new	short	throw
break	char	double	float	instanceof	package	static	throws
byte	class	else	for	int	private	super	try
	continue	enum		interface	protected	switch	void
		extends			public	synchronized	while

1.1 Operators

Arithmetic and assignment operators

Symbol	Operator	Written as	Meaning
*	multiplication	$a * b$	a times b
/	division	a / b	a divided by b
%	remainder	$a \% b$	the remainder of (a divided by b)
+	addition	$a + b$	a added to b
-	subtraction	$a - b$	a minus b
++	postfix increment	$x++$	increments x by 1 and returns the old value
++	prefix increment	$++x$	increments x by 1 and returns the new value
--	postfix decrement	$x--$	decrements x by 1 and returns the old value
--	prefix decrement	$--x$	decrements x by 1 and returns the new value
=	assign	$a = b$	put the value of b into a
+=	add and assign	$a += b$	put the value of $a + b$ into a
-=	subtract and assign	$a -= b$	put the value of $a - b$ into a
*=	multiply and assign	$a *= b$	put the value of $a * b$ into a
/=	divide and assign	$a /= b$	put the value of a / b into a
%=	remainder and assign	$a \% = b$	put the value of $a \% b$ into a

Logical and relational operators

Symbol	Operator	Written as	Meaning
&&	Logical and	<code>a && b</code>	returns true if both <code>a</code> and <code>b</code> are true, otherwise false
	Logical or	<code>a b</code>	returns false if both <code>a</code> and <code>b</code> are false, otherwise true
!	Logical negation	<code>!a</code>	returns false if <code>a</code> is true, returns true if <code>a</code> is false
==	equal to	<code>x == y</code>	true if <code>x</code> equals <code>y</code> , otherwise false
>	greater than	<code>x > y</code>	true if <code>x</code> is greater than <code>y</code> , otherwise false
<	less than	<code>x < y</code>	true if <code>x</code> is less than <code>y</code> , otherwise false
>=	greater than or equal to	<code>x >= y</code>	true if <code>x</code> is greater than or equal to <code>y</code> , otherwise false
<=	less than or equal to	<code>x <= y</code>	true if <code>x</code> is less than or equal to <code>y</code> , otherwise false
!=	not equal to	<code>x != y</code>	true if <code>x</code> is not equal to <code>y</code> , otherwise false

1.2 Iteration

Keyword	Meaning	Example of use
for	A looping statement indicating a starting value of a control variable, a condition to evaluate that determines when the loop should end, and an adjustment to the control variable, together with statements to be performed while the loop condition holds.	<pre>for (int j = 1; j < 10; j++) { System.out.println(j + " "); }</pre>
	A variant form of for-loop called for-each iterates over the objects contained in a collection implementing <code>Iterable</code> , or an array, and extracts each of them. If the collection guarantees ordering, that order is used.	<pre>for (Pupil p : pupilCollection) { if (p.getAge() > 10) { System.out.println(p); } }</pre>
while	A looping statement, the body of which will be repeatedly executed as long as some condition continues to evaluate to true.	<pre>while (countdown != 0) { System.out.println(countdown); countdown--; }</pre>
	A variant form of while begins with the keyword <code>do</code> . In this case, the condition is only checked after the first execution of the body.	<pre>do { System.out.println(countdown); countdown--; } while(countdown > 1);</pre>

1.3 Selection

Keyword	Meaning	Example of use
if	A selection statement, allowing the flow of program control to be changed. A logical condition is evaluated and a block of code executed only if the condition evaluates to <code>true</code> .	<pre>if (day == 0) { System.out.println("Sunday"); }</pre>
	Optionally the keyword <code>else</code> can be used to specify code to be executed if the condition evaluates to <code>false</code> .	<pre>if (num % 2 == 0) { System.out.println("even"); } else { System.out.println("odd"); }</pre>
switch	This keyword is used for a selection statement allowing choice of one of several paths of program control depending on the value of an argument (here <code>val</code>) of primitive, <code>enum</code> or wrapper type.	<pre>switch (val) { case 'a': { videoId = 3; break; } case 'b': { videoId = 19; break; } default: { System.out.println("42"); break; } }</pre>

1.4 Arrays

Aspects	Example of use
A linear form of storage, indexed by an <code>int</code> beginning from 0. An array can be declared using the type it stores, followed by square brackets.	<pre>int[] ia; //array of ints String[] sa; //array of String refs</pre>
Arrays are instantiated using the keyword <code>new</code> and then the type, with a size argument.	<pre>ia = new int[4]; //room for 4 ints</pre>
Default values are used to initialize an array if no explicit initialization is performed. You can also use an array initializer when creating an array.	<pre>char[] myChars = {'a', 'c', 'x'};</pre>
Array contents are accessed using an <code>int</code> index.	<pre>char f = myChars[0];</pre>
An array's length can be found using its <code>length</code> instance data.	<pre>int len = ia.length;</pre>

1.5 Exceptions

Keyword	Meaning	Example of use
<code>try</code>	Introduces a block of code in which an exception can occur.	<pre>try { //something }</pre>
<code>catch</code>	Follows a try block and introduces code to handle an exception of the specified type.	<pre>catch(Exception ex) { System.out.println("Caught exception ex.getMessage()); }</pre>
<code>finally</code>	Introduces a block of code that will always be performed, after a try-catch statement.	<pre>finally { //always do something }</pre>
<code>throws</code>	Used to Indicate that one or more exceptions (separated by commas) may be thrown by a method or constructor.	<pre>throws Exception</pre>
<code>throw</code>	Used to throw an exception.	<pre>throw new Exception();</pre>

2 Java API excerpts

2.0 Errors and exceptions

The following table includes a number of common exception types.

Class	Meaning
<code>ArithmeticException</code> extends <code>RunTimeException</code>	Thrown when an illegal arithmetic condition occurs for integral types.
<code>ArrayIndexOutOfBoundsException</code> extends <code>RunTimeException</code>	Thrown by an attempt to access an element in an array which is outside its declared boundaries.
<code>EOFException</code> extends <code>IOException</code>	Thrown on attempting to read past the end of a file or stream.
<code>Exception</code> extends <code>Throwable</code>	Top-level class of checked exceptions.
<code>FileNotFoundException</code> extends <code>IOException</code>	Thrown on attempting to open or write to a file and the file is not found.
<code>IOException</code> extends <code>Exception</code>	General class of exceptions due to failed or interrupted input and output operations.
<code>MalformedURLException</code> extends <code>IOException</code>	Thrown on constructing an incorrectly formed URL.
<code>NullPointerException</code> extends <code>RunTimeException</code>	Thrown on an attempt to access an object from a null reference.
<code>RunTimeException</code> extends <code>Exception</code>	Top-level class of exceptions that may be thrown by the virtual machine at runtime (unchecked).
<code>Throwable</code> extends <code>Object</code>	Top-level class of all errors and exceptions (unchecked and checked).
<code>UnsupportedOperationException</code> extends <code>RunTimeException</code>	Thrown to indicate that the requested operation is not supported; for example, if an optional interface operation is not supported.
<code>Error</code> extends <code>Throwable</code>	Top-level class of exceptions that occur when some internal Java error has happened – for example, the Java system has run out of memory (unchecked).

2.1 Some common methods of the `Thread` class

The Java Virtual Machine allows an application to have multiple threads of execution running concurrently. Threads are instances of the `Thread` class.

Sample methods	Meaning
<code>String getName()</code>	Returns this thread's name.
<code>int getPriority()</code>	Returns this thread's priority.
<code>void join()</code>	Waits for this thread to die.
<code>void run()</code>	If this thread was constructed using a separate <code>Runnable</code> object, then that <code>Runnable</code> object's <code>run</code> method is called; otherwise, this method does nothing and returns.
<code>void setName(String name)</code>	Changes the name of this thread to be equal to the argument <code>name</code> .
<code>void setPriority(int priority)</code>	Changes the priority of this thread.
<code>static void sleep(long millis)</code>	Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds.
<code>void start()</code>	Causes this thread to begin execution; the Java Virtual Machine calls the <code>run</code> method of this thread.
<code>String toString()</code>	Returns a string representation of this thread, including the thread's name, priority, and thread group.
<code>static void yield()</code>	Causes the currently executing thread object to temporarily pause and allow other threads to execute.

2.2 Some common methods of the `String` class

Sample Methods	Meaning
<code>char charAt(int i)</code>	Finds and returns the character at position <code>i</code> .
<code>boolean equals(Object o)</code>	Compares this string to the specified object.
<code>int indexOf(int ch)</code>	Returns the index within this string of the first occurrence of the specified character or <code>-1</code> if not found.
<code>int indexOf(String str)</code>	Searches for a particular string <code>str</code> within the destination object and returns the position of the first index of <code>str</code> within the string. The search starts at the front of the string.
<code>int lastIndexOf(String str)</code>	This is the same as <code>indexOf</code> but searches from the back of the destination string object.
<code>int length()</code>	Returns the length of the string.
<code>String substring(int beginIndex, int endIndex)</code>	This returns a substring starting at position <code>beginIndex</code> and ends at position <code>endIndex - 1</code> of the destination string object.
<code>String toLowerCase()</code>	Returns the characters in this <code>String</code> in lower case using the rules of the default locale.
<code>String toUpperCase()</code>	Returns the characters in this <code>String</code> in upper case using the rules of the default locale.
<code>String valueOf(x)</code>	Returns a string representation of the argument <code>x</code> , which may be of any primitive type, or of type <code>Object</code> .

2.3 Some important interfaces

In some cases* we have listed only the more commonly required methods defined by the interface. The list of classes that implement the interfaces are not intended to be complete.

Interface	Sample methods	Implemented by
<code>Collection<E></code> <code>extends Iterable<E></code> * The root interface in the collection hierarchy representing a group of objects, known as its elements.	<code>boolean add(E o)</code> <code>boolean contains(Object o)</code> <code>boolean isEmpty()</code> <code>Iterator<E> iterator()</code> <code>boolean remove(Object o)</code> <code>int size()</code> <code>Object[] toArray()</code>	The JDK provides implementations only of more specific subinterfaces like <code>Set</code> and <code>List</code> .
<code>Comparable<T></code> Imposes a total ordering on the objects of each class that implements it.	<code>int compareTo(T o)</code> Compares this object with the specified object for order.	<code>Boolean</code> <code>Byte</code> <code>Character</code> <code>Double</code> <code>Float</code> <code>Integer</code> <code>Long</code> <code>Short</code> <code>String</code>
<code>Iterable<T></code> Implementing this interface allows an object to be the target of a for-each statement.	<code>Iterator<T> iterator()</code>	<code>ArrayList</code> <code>HashSet</code> <code>LinkedList</code> <code>TreeSet</code>
<code>Iterator<E></code> An iterator over a collection. The <code>remove</code> operation is optional; that is, a class can throw <code>UnsupportedOperationException</code> if <code>remove</code> is not implemented.	<code>boolean hasNext()</code> <code>E next()</code> <code>void remove()</code>	<code>Scanner</code>
<code>List<E></code> <code>extends Collection<E></code> * An ordered collection.	In addition to <code>Collection</code> methods, <code>E get(int index)</code> <code>int indexOf(Object o)</code> <code>E set(int index, E element)</code>	<code>ArrayList</code> <code>LinkedList</code>
<code>Map<K, V></code> * An object that maps keys to values.	<code>boolean containsKey(Object key)</code> <code>boolean containsValue(Object value)</code> <code>V get(Object key)</code> <code>V put(K key, V value)</code> <code>int size()</code>	<code>HashMap</code> <code>TreeMap</code>
<code>Runnable</code> The <code>Runnable</code> interface should be implemented by any class whose instances are intended to be executed by a thread.	<code>void run()</code>	<code>Thread</code>

Interface	Sample methods	Implemented by
<code>Serializable</code> Implementing this interface enables serializability of a class. Classes that do not implement this interface will not have any of their state serialized or deserialized. All subtypes of a serializable class are themselves serializable.	The serialization interface has no methods or fields and serves only to identify the semantics of being serializable.	<code>ArrayList</code> <code>HashMap</code> <code>HashSet</code> <code>LinkedList</code> <code>TreeMap</code> <code>TreeSet</code>
<code>Set<E></code> extends <code>Collection<E></code> * A collection that contains no duplicate elements.	In addition to <code>Collection</code> methods, <code>boolean equals(Object o)</code> <code>boolean isEmpty()</code>	<code>HashSet</code> <code>TreeSet</code>
<code>SortedMap<K, V></code> extends <code>Map<K, V></code> * A map that further guarantees that it will be in ascending key order according to the natural ordering of its keys (see <code>Comparable</code>).	In addition to <code>Map</code> methods, <code>K firstKey()</code> <code>K lastKey()</code>	<code>TreeMap</code>
<code>SortedSet<E></code> extends <code>Set<E></code> * A set that further guarantees that its iterator will traverse the set in ascending element order, sorted according to the natural ordering of its elements (see <code>Comparable</code>).	In addition to <code>Set</code> methods, <code>E first()</code> <code>E last()</code> <code>SortedSet<E> subSet(E fromEl, E toEl)</code>	<code>TreeSet</code>

2.4 Collection classes

Legacy data structures such as `Hashtable` and `Stack` and `Vector` have been omitted.

Collection class	Meaning	Implements
<code>ArrayList<E></code>	Can store a variable number of references, similar to an array.	<code>Collection<E></code> <code>Iterable<E></code> <code>List<E></code> <code>Serializable</code>
<code>HashMap<K, V></code>	Hash table based implementation of the <code>Map</code> interface.	<code>Map<K, V></code> <code>Serializable</code>
<code>HashSet<E></code>	Implements the <code>Set</code> interface, backed by a hash table.	<code>Collection<E></code> <code>Iterable<E></code> <code>Serializable</code> <code>Set<E></code>
<code>LinkedList<E></code>	Linked list implementation of the <code>List</code> interface.	<code>Collection<E></code> <code>Iterable<E></code> <code>List<E></code> <code>Serializable</code>
<code>TreeMap<K, V></code>	Tree based implementation of the <code>SortedMap</code> interface.	<code>Map<K, V></code> <code>Serializable</code> <code>SortedMap<K, V></code>
<code>TreeSet<E></code>	Implements the <code>Set</code> and the <code>SortedSet</code> interfaces, backed by a <code>TreeMap</code> .	<code>Collection<E></code> <code>Iterable<E></code> <code>Serializable</code> <code>Set<E></code> <code>SortedSet<E></code>

2.5 Input and output streams and related classes

A stream is a sequence of bytes and the various stream classes provide ways of interacting with such streams. We have only listed the more commonly used input and output streams.

Classes whose names end in `Stream` handle raw data in terms of bytes. Bytes are read as `int` values in the range 0 to 255. If no byte is available because the end of the stream has been reached, `-1` is returned.

Classes whose names end in `Reader` or `Writer` handle character data, represented as an `int`.

Stream methods may throw an `IOException`.

The remainder of this document is printed in a landscape format to accommodate some wide tables.

2.5.1 Input streams

We list here some of the more commonly used input streams.

InputStream and its descendants

Stream	Sample methods and comments	Constructors
InputStream The abstract superclass of all classes representing an input stream of bytes. Methods listed here are also available to subclasses (but may have been overridden).	<code>void close()</code> <code>abstract int read()</code> <code>int read(byte[] b)</code>	<code>InputStream()</code>
FileInputStream Obtains bytes from a file.	see <code>InputStream</code>	<code>FileInputStream(File f)</code> <code>FileInputStream(String name)</code>
BufferedInputStream Adds ability to buffer input to another input stream.	Adds methods to mark a place in a stream and return to it, while the <code>readLimit</code> is not exceeded. <code>void mark(int readlimit)</code> <code>void reset()</code>	<code>BufferedInputStream(InputStream in)</code>
DataInputStream Reads from an underlying input stream and converts to primitive types.	Adds methods to read primitive types, for example: <code>float readFloat()</code> <code>boolean readBoolean()</code>	<code>DataInputStream(InputStream in)</code>

Reader and its descendants

Classes based on `Reader` are for reading character streams.

A read character is returned as an `int` value.

Stream	Sample methods and comments	Constructors
<code>Reader</code> Abstract class for reading character streams. Methods listed here are also available to subclasses (but may have been overridden).	<code>abstract void close()</code> <code>int read()</code>	<code>protected Reader()</code>
<code>BufferedReader</code> Read text from a character-input stream, buffering characters.	Adds methods to mark a place in a stream and return to it, while the <code>readLimit</code> is not exceeded. <code>void mark(int readLimit)</code> <code>String readLine()</code> <code>void reset()</code>	<code>BufferedReader(Reader in)</code> <code>BufferedReader(Reader in, int bufSiz)</code>
<code>InputStreamReader</code> A bridge from byte streams to character streams.	see <code>Reader</code>	<code>InputStreamReader (InputStream in)</code>
<code>FileReader</code> Convenience class for reading character files.	see <code>Reader</code>	<code>FileReader(File f)</code> <code>FileReader(String name)</code>

2.5.2 Output streams

Operations may throw an `IOException`. Output streams may be flushed.

`OutputStream` and its descendants

A character to be written is contained in the 16 low-order bits of a given integer value; the 16 high-order bits are ignored. (So, an `int` represents a single character to be written.)

Stream	Sample methods and comments	Constructors
<code>OutputStream</code> The abstract superclass of all classes representing an output stream of bytes. Methods listed here are also available to subclasses (but may have been overridden).	<code>void close()</code> <code>void flush()</code> <code>void write(byte[] b)</code> <code>void write(int b)</code>	<code>OutputStream()</code>
<code>FileOutputStream</code> extends <code>OutputStream</code> An output stream for writing bytes to a file.	See <code>OutputStream</code>	<code>FileOutputStream(File f)</code> <code>FileOutputStream(String name)</code> <code>FileOutputStream(String name, boolean append)</code>
<code>DataOutputStream</code> extends <code>OutputStream</code> Writes primitive types to an underlying output stream.	Adds methods for writing primitive types, for example: <code>void writeInt(int v)</code> <code>void writeFloat(float v)</code>	<code>DataOutputStream (OutputStream out)</code>
<code>BufferedOutputStream</code> extends <code>FilterOutputStream</code> Adds buffering to another output stream.	See <code>OutputStream</code>	<code>BufferedOutputStream(OutputStream out)</code>
<code>PrintStream</code> extends <code>FilterOutputStream</code> Adds ability to print representations of various data values.	Adds <code>print</code> and <code>println</code> methods for primitive data types, for example: <code>print (boolean b)</code> <code>println(char c)</code>	<code>PrintStream (File f)</code> <code>PrintStream(OutputStream out)</code> <code>PrintStream(OutputStream out, boolean autoflush)</code> <code>PrintStream(String filename)</code>

Writer and its descendants

Stream	Sample methods and comments	Constructors
Writer Abstract class for writing to character streams. Methods listed here are also available to subclasses (but may have been overridden).	abstract void close() abstract void flush() void write(int c) void write(String s)	protected Writer()
BufferedWriter Writes text to a character-output stream, buffering characters.	See Writer	BufferedWriter(Writer out)
OutputStreamWriter A bridge from character streams to byte streams.	See Writer	OutputStreamWriter(OutputStream out)
PrintWriter Prints formatted representations of objects to a text-output stream.	Adds print and println methods for primitives and String, for example: print(int b) println(String s)	PrintWriter (File f) PrintWriter(OutputStream out) PrintWriter(OutputStream out, boolean autoflush) PrintWriter(String filename) PrintWriter(Writer out)
FileWriter Convenience class for writing character files.	See Writer	FileWriter (File f) FileWriter(File f, boolean append) FileWriter(String filename)

2.5.3 Standard streams

Stream	Comments	Examples
<code>System.in</code> Standard input, normally the keyboard.	<code>in</code> is a static <code>InputStream</code> in the <code>System</code> class. This stream is already open and ready to supply input data.	<code>Scanner scnr = new Scanner(System.in)</code>
<code>System.out</code> Standard output, normally the screen console.	<code>out</code> is a static <code>PrintStream</code> in the <code>System</code> class. This stream is already open and ready to accept output data.	<code>System.out.println("Off they go!");</code>
<code>System.err</code> Standard error stream, normally the screen console.	<code>err</code> is a static <code>PrintStream</code> in the <code>System</code> class. This stream is already open and ready to accept output data.	<code>System.err.println("bad wolf");</code>

2.5.4 Scanner

Stream	Sample methods and comments	Sample constructors
Scanner Implements the <code>Iterator<String></code> interface and can read text from files, input streams, strings or any object that implements the <code>Readable</code> interface. The default delimiter for tokens is whitespace.	<code>boolean hasNext()</code> <code>String next()</code> //next token <code>String nextLine()</code> Similar methods are provided for primitive types, for example: <code>int nextInt()</code> <code>boolean hasNextInt()</code>	<code>Scanner(File source)</code> <code>Scanner(InputStream source)</code> <code>Scanner(String source)</code>

2.5.5 Sockets

Class	Sample methods	Sample constructors
<code>ServerSocket</code> Implements a server socket which waits for requests to come in over a network.	<code>Socket accept()</code> <code>void close()</code> <code>int getLocalPort()</code> <code>String toString()</code>	<code>ServerSocket(int port)</code>
<code>Socket</code> Implements a client socket; an endpoint for communication between two machines.	<code>InputStream getInputStream()</code> <code>OutputStream getOutputStream()</code> <code>int getPort()</code> <code>void close()</code>	<code>Socket(String host, int port);</code>

2.6 Java Swing classes

Note that most widgets have many variant constructors and methods and space would not permit listing them all. In the examples below we have simply picked the commonest constructors for each widget and some of the frequently used methods.

Top-level Swing containers – such as `JFrame`, `JDialog` and `JApplet` – are specialized components that provide a place for other Swing components to paint themselves. These classes inherit from the `Container` class, and provide methods to add components, with or without constraints (such as can be applied to a `BorderLayout`), as well as a method to remove a component.

See also the event-handling classes in Section 2.7.

Class	Sample methods	Sample constructors
<code>ButtonGroup</code> A group of radio buttons.	<code>Component add(JRadioButton j)</code> <code>void remove(JRadioButton j)</code>	<code>ButtonGroup ()</code>
<code>JApplet</code> An applet container with support for Swing component architecture.	<code>void add(Component c)</code> <code>void add(Component c, Object constraints)</code> <code>void init()</code> <code>void remove(Component c)</code> <code>void setLayout(LayoutManager m)</code> <code>void start()</code> <code>void stop()</code> <code>void destroy()</code>	<code>JApplet ()</code> Default layout for content pane is <code>BorderLayout</code>
<code>JButton</code> An implementation of a button that can be clicked.	<code>String getText()</code> <code>void setText(String text)</code>	<code>JButton ()</code> <code>JButton(String text)</code>

Class	Sample methods	Sample constructors
JCheckBox An implementation of a check box that can be selected or deselected.	Object[] getSelectedObjects() String getText() boolean isSelected() void setSelected(boolean b)	JCheckBox (String text)
JComboBox A component that combines a button or editable field and a drop-down list.	void addItem(Object o) Object getItemAt(int index) Object getSelectedItem() Object[] getSelectedObjects() void setSelectedItem(Object o)	JComboBox() JComboBox (Object[] items)
JFrame A top-level container; the window used in a graphical user interface.	void add(Component c) void add(Component c, Object constraints) Container getContentPane() void paint(Graphics g) void remove(Component c) void repaint() void setDefaultCloseOperation(int operation) void setMenuBar(JMenuBar m) void setLayout(LayoutManager m) void setTitle(String title) void setVisible(boolean value) void update(Graphics g)	JFrame (String title) Default layout is BorderLayout
JLabel A display area for a short text string.	String getText() void setText(String text)	JLabel (String text)

Class	Sample methods	Sample constructors
JList A component that allows the user to select one or more objects from a list. Occupies a fixed number of lines.	Object[] getSelectedValues() int[] getSelectedIndices()	JList (Object[] listData);
JMenu An implementation of a pull-down menu that can be held in a JMenuBar.	JMenuItem add(JMenuItem j) JMenuItem add(String s) void insert (String s, int pos) void remove (int pos)	JMenu (String text)
JMenuBar An implementation of a menu bar acting as a holder for menus.	JMenu add(JMenu c)	JMenuBar ()
JMenuItem An implementation of an item in a menu.	void add(Component c) void init(String text, Icon icon) void setEnabled(boolean b)	JMenuItem (String text)
JPanel A container used to place widgets and which can be added to a JFrame or to another panel.	Component add(Component c) void add(Component c, Object constraints) void paintComponent(Graphics g) void remove(Component c) void repaint() void setLayout (LayoutManager m)	JPanel () JPanel (LayoutManager layout) Default layout is FlowLayout

Class	Sample methods	Sample constructors
JRadioButton An implementation of a radio button, used in conjunction with a ButtonGroup so that only one radio button at once can be selected.	void setText(String text) boolean isSelected() void setSelected(boolean b)	JRadioButton (String text) JRadioButton (String text, boolean selected)
JScrollBar An implementation of a scroll bar with a slider that can be moved.	int getMinimum() int getMaximum() int getValue() int setMinimum()	JScrollBar (); JScrollBar (int orientation);
JScrollPane A scrolling pane that includes horizontal and vertical scroll bars and can contain a list or text area.	Used to add scrolling ability to a Component such as a JList or JTextArea.	JScrollPane (Component c)
JTextArea An implementation of a multi-line holder of text.	void append(String s) int getLineCount() void setColumns(int c) void setRows(int r) void setText(String s)	JTextArea(String s) JTextArea (String s, int rows, int cols) JTextArea (int rows, int cols)
JTextField An implementation of a single line holder of text.	String getText() void setText(String s)	JTextField() JTextField(int columns) JTextField(String text)

2.7 Event handling

Relevant components have an add method formed from the name of the listener, for example, addActionListener or addAdjustmentListener. The ActionEvent class provides a method Object getSource() method to return a reference to the object that produced an event.

Interface	Methods	Events generated by the following
ActionListener	void actionPerformed(ActionEvent e)	Buttons, lists, menu items and text fields.
AdjustmentListener	void adjustmentValueChanged(AdjustmentEvent e)	Scroll bars.
ComponentListener	void componentHidden(ComponentEvent e) void componentMoved(ComponentEvent e) void componentResized(ComponentEvent e) void componentShown(ComponentEvent e)	Visual components; for example, being resized or hidden.
ContainerListener	void componentAdded(ContainerEvent e) void componentRemoved(ContainerEvent e)	Containers such as frames; for example, when a component is added or removed.
FocusListener	void focusGained(FocusEvent e) void focusLost(FocusEvent e)	Components coming into focus or going out of focus.
ItemListener	void itemStateChanged(ItemEvent e)	Check boxes, choices and lists.
KeyListener	void keyPressed(KeyEvent e) void keyReleased(KeyEvent e) void keyTyped(KeyEvent e)	Keys being pressed or released.
MouseListener	void mouseClicked(MouseEvent e) void mouseEntered(MouseEvent e) void mouseExited(MouseEvent e) void mousePressed(MouseEvent e) void mouseReleased(MouseEvent e)	Actions such as clicking or moving a mouse.
TextListener	void textValueChanged(TextEvent e)	Text components such as text fields and text areas.
WindowListener	void windowClosing(WindowEvent e)	Windows being opened or closed.

2.8 Layout managers

Layout manager	Effect	Sample code
absolute positioning	Components are placed manually. The layout manager is specified to be <code>null</code> . <code>setBounds</code> is used to place and size the component.	<pre>holder.setLayout(null); JButton jb = new JButton(); holder.add(jb); jb.setBounds(10,10,20,20);</pre>
BorderLayout	Components are placed at north, south, east, west, or centre.	<pre>BorderLayout() data includes static constants BorderLayout.NORTH BorderLayout.SOUTH BorderLayout.WEST BorderLayout.EAST</pre>
FlowLayout	Components are arranged like words in a paragraph, flowing to the next line if they will not fit on the current one.	<pre>FlowLayout()</pre>
GridLayout	Components are arranged in a grid of rows and columns (arguments are in that order).	<pre>GridLayout(int rows, int cols)</pre>

2.9 Graphics

Sample methods	Meaning
<code>drawRect(int x, int y, int width, int height)</code>	Draws the outline of the specified rectangle. The left and right edges of the rectangle are at <code>x</code> and <code>x + width</code> . The top and bottom edges are at <code>y</code> and <code>y + height</code> . The rectangle is drawn using the graphics context's current colour.
<code>drawLine(int x1, int y1, int x2, int y2)</code>	Draws a line, using the current colour, between the points <code>(x1, y1)</code> and <code>(x2, y2)</code> in this graphics context's coordinate system.
<code>drawOval(int x, int y, int width, int height)</code>	Draws the outline of an oval. The result is a circle or ellipse that fits within the rectangle specified by the <code>x</code> , <code>y</code> , <code>width</code> , and <code>height</code> arguments.
<code>drawString(String s, int x, int y)</code>	Renders the text of the specified <code>String</code> , using the current text attribute in the graphics context, starting from <code>(x, y)</code>
<code>fillOval(int x, int y, int width, int height)</code>	Fills an oval bounded by the specified rectangle with the current colour.
<code>void setColor(Color c)</code>	Sets this graphics context's current colour to the specified colour. All subsequent graphics operations using this graphics context use this specified colour. Colours are static constants, for example <code>Color.GREEN</code> , <code>Color.RED</code> etc.

2.10 Some common methods of the MIDlet class

Method	Meaning
<code>protected abstract void startApp()</code>	Signals the <code>MIDlet</code> that it has entered the <i>Active</i> state from the <i>Paused</i> state.
<code>protected abstract void pauseApp()</code>	Signals the <code>MIDlet</code> to enter the <i>Paused</i> state from the <i>Active</i> state.
<code>protected abstract void destroyApp(Boolean b)</code>	Signals the <code>MIDlet</code> to terminate, releasing its resources, and enter the <i>Destroyed</i> state.